MODULE 02 · DOMAIN 1 - SDLC AUTOMATION

Pipeline Anatomy

How CodePipeline starts, what it runs, how it queues, and how it decides to stop, skip or roll back a stage.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 2 of 34

admin@sthithapragna.com

A pipeline is a conveyor belt for **revisions**. Most questions are really asking what happens when two revisions are on the belt at once.

Pipeline, stage, action, artifact

01
Pipeline
The whole release process, one per service or repo.

02
Stage
A named step. One execution at a time, unless PARALLEL.

03
Action
The work: build, test, deploy, approve, invoke.

04
Artifact
Files passed between actions, stored in S3.

05
Variable
Values passed between actions, such as a commit ID.

ON THE EXAM

Actions with the same run order inside a stage run in parallel. Different run orders run in sequence.

Action categories, and what plugs into each

CATEGORY	COMMON PROVIDERS
Source	CodeConnections (GitHub, GitLab, Bitbucket), CodeCommit, S3, ECR
Build	CodeBuild, Commands, ECRBuildAndPublish
Test	CodeBuild, Device Farm, third-party tools
Deploy	CodeDeploy, ECS, CloudFormation, Elastic Beanstalk, S3, Lambda, EC2
Approval	Manual approval
Invoke	Lambda, Step Functions, another CodePipeline pipeline

ON THE EXAM

If a scenario needs a step no provider covers, the answer is usually an Invoke action or CodeBuild running a script.

V1 or V2?

V2 TYPE

The current pipeline

- Git triggers with branch, tag and path filters
- QUEUED and PARALLEL execution modes
- Pipeline-level variables, stage conditions
- $0.002 per action-execution minute

V1 TYPE

The original pipeline

- Starts on any change to one branch
- SUPERSEDED mode only
- No stage conditions or pipeline variables
- $1 per active pipeline per month

THE TIEBREAKER

If a scenario needs any feature in the left column, the first step is converting the pipeline to V2.

How each source starts a pipeline

SOURCE	CHANGE DETECTION
GitHub, GitLab, Bitbucket	Webhook through CodeConnections. V2 triggers can filter it
CodeCommit	EventBridge rule on the repository
S3 object	EventBridge, with the bucket versioned and events recorded
ECR image	EventBridge rule on the image push
Anything else	Manual start, EventBridge schedule, or StartPipelineExecution

ON THE EXAM

Periodic polling is the legacy option for every source. Event-based detection is the answer the exam expects.

A V2 trigger, read line by line

```
1  "triggers": [{
2    "providerType": "CodeStarSourceConnection",
3    "gitConfiguration": {
4      "sourceActionName": "Source",
5      "push": [
6        { "branches":  { "includes": ["main"] },
7          "filePaths": { "includes": ["services/payments/**"] } },
8        { "tags": { "includes": ["v*"], "excludes": ["v*-rc*"] } }
9      ],
10     "pullRequest": [
11       { "events": ["OPEN", "UPDATED"], "branches": { "includes": ["main"] } }
12     ] } }]
```

6-7 One filter: main AND a path under payments. Both must match.

8 A second filter: release tags, excluding release candidates. Either filter starts a run.

ON THE EXAM

Filters are OR'd with each other. Conditions inside one filter are AND'd. Excludes always beat includes.

Trigger behaviour worth knowing

- A **push** run uses the commit on the branch pushed to. A **pull request** run uses the commit on the source branch.
- Adding any trigger **disables the default** change detection on the action's branch.
- A new branch's **first push** does not match a file-path filter: there is no list of changed files yet.
- Merging a pull request can start **two runs** if push and pull request filters overlap.
- Triggers exist only for **connection-based** sources. CodeCommit and S3 filter through EventBridge rules instead.

> ON THE EXAM
>
> A monorepo where one change starts every service's pipeline is solved by file-path filters, not by custom code.

Three execution modes

MODE	WHEN A NEWER RUN ARRIVES	USE IT WHEN
SUPERSEDED	The older run finishes its stage, then the newer one replaces it	Only the latest change matters
QUEUED	The newer run waits its turn; every run completes in order	Every change must deploy, in order
PARALLEL	Runs travel independently; stages are not locked	Fast, isolated feedback, such as pull requests

ON THE EXAM

SUPERSEDED is the default, and it can skip revisions entirely. That is a feature until a skipped commit carried a migration.

Picking the mode from the wording

CHOOSE QUEUED WHEN

Order and completeness matter

- Each deployment must finish before the next
- Every merged commit must reach production
- Infrastructure changes build on one another
- Typical for a main-branch release pipeline

CHOOSE PARALLEL WHEN

Speed of feedback matters

- Results for every pull request, quickly
- Runs target separate, isolated environments
- No shared state between runs
- Typical for a pull request pipeline

THE TIEBREAKER

Two pipelines, two modes: PARALLEL for pull requests, QUEUED for main. SUPERSEDED when 'only the latest' is stated.

Variables: passing values, not files

- **Source variables:** CommitId, BranchName, AuthorDate, and more, from the source action.
- **Action output variables** live in a **namespace** and are referenced as #{Namespace.Name}.
- CodeBuild **exports** variables from its buildspec for later actions to use.
- **Pipeline-level variables** (V2) take a value when the run starts, with a default.

> ON THE EXAM
>
> Artifacts carry files. Variables carry small values such as an image tag or a commit ID. Do not zip a single string.

An image tag, from build to deploy

```
1  # buildspec.yml
2  env:
3    exported-variables: [IMAGE_TAG]
4  phases:
5    build:
6      commands:
7        - export IMAGE_TAG=$(echo $CODEBUILD_RESOLVED_SOURCE_VERSION | cut -c1-8)
8
9  # later action, configuration field in the pipeline
10 "ImageTag": "#{BuildVariables.IMAGE_TAG}"
```

3 Declares which environment variable leaves the build.

10 BuildVariables is the namespace given to the build action.

ON THE EXAM

No namespace on the build action, no way to reference its variables. That is the usual reason the value arrives empty.

Manual approval

- Pauses the run. Can notify an **SNS topic** and include a **URL** for reviewers to check.
- Approvers need **codepipeline:PutApprovalResult** on that pipeline. Comments are recorded.
- Default timeout **7 days**. Now configurable per action, **5 minutes to 60 days**.
- It proves someone agreed. It cannot detect a runtime fault that has not happened yet.

> ON THE EXAM
>
> Change control or sign-off: approval action. Automatic protection from bad releases: alarms and rollback.

Stage conditions: three moments, several results

CONDITION	CHECKED	POSSIBLE RESULTS
Entry	Before the stage starts	Fail, or Skip
On Failure	When the stage has failed	Roll back, or Retry
On Success	After the stage has succeeded	Roll back, or Fail

ON THE EXAM

Each condition holds a set of rules. If any rule fails, the condition fails and its result engages.

The rules a condition can use

RULE	IT CHECKS	TYPICAL USE
`CloudWatchAlarm`	Is an alarm in ALARM?	Block or roll back on bad health
`DeploymentWindow`	Is now inside a cron window?	Only deploy in business hours
`VariableCheck`	Does a variable match?	Skip a stage for some branches
`LambdaInvoke`	Does a function return success?	Any custom check
`CodeBuild`	Does a build succeed?	Integration tests after deploy
`Commands`	Do shell commands exit zero?	A quick smoke test

ON THE EXAM

Only VariableCheck and LambdaInvoke can drive a Skip result. CodeBuild and Commands rules arrived in March 2025.

A health check after deploy, without code

```
"name": "Production",
"actions": [ ... ],
"onSuccess": {
  "conditions": [{
    "result": "ROLLBACK",
    "rules": [{
      "name": "ErrorRateAfterDeploy",
      "ruleTypeId": { "category": "Rule", "owner": "AWS",
                      "provider": "CloudWatchAlarm", "version": "1" },
      "configuration": { "AlarmName": "prod-5xx-rate", "WaitTime": "30" }
    }] }] }
```

5 If the rule fails, return the stage to the last successful execution.

10 If the alarm is firing when checked, allow 30 minutes for it to clear first.

ON THE EXAM

WaitTime is a grace period for an alarm already firing, not a watch window. If the alarm is OK, the rule passes at once.

Rollback and retry

- **Stage rollback** re-runs the stage with the source revisions of its last successful execution.
- Rollback only reaches executions from the **current pipeline structure**. Edit the pipeline, lose the history.
- **Retry** re-runs the failed actions, or the whole failed stage, on the same revision.
- Rollback redeploys an older artifact. It does **not** revert your Git history.

> ON THE EXAM
>
> Transient failure, same code: retry. Bad code that deployed: roll back, then fix forward in Git.

Two rollbacks at two levels

INSIDE THE DEPLOY ACTION

CodeDeploy or ECS rollback

- Alarm fires during the traffic shift
- Traffic returns within the same deployment
- Configured on the deployment group or service
- Catches problems while traffic is moving

AROUND THE STAGE

CodePipeline stage rollback

- Evaluated when the stage completes
- Re-runs the stage with the last good revision
- Configured as a stage condition
- Catches a system unhealthy once it lands

THE TIEBREAKER

Problem while traffic is moving: deployment-level rollback. Unhealthy once the stage completes: a stage condition.

Invoking custom work

- A **Lambda invoke** action must call **PutJobSuccessResult** or **PutJobFailureResult**.
- With no reply after **20 minutes**, the action times out and fails, even if the function succeeded.
- A **Step Functions** invoke action suits longer, multi-step work. Default timeout 7 days.
- The **Commands** action runs shell commands on managed compute without a CodeBuild project.

> ON THE EXAM
>
> A Lambda action 'stuck in progress' with a successful function log is almost always a missing job-result call.

Watching the pipeline itself

- Pipeline, stage and action **state changes** are sent to EventBridge automatically.
- A rule on FAILED can notify **SNS**, open an OpsCenter item, or start a runbook.
- **Notification rules** in the console send the same events to SNS or chat channels.
- CloudTrail records who **changed** or started a pipeline. It is not how you watch runs.

> ON THE EXAM
>
> 'Notify the team when any production deployment fails' is an EventBridge rule on the pipeline state change event.

A company deploys database schema changes and application code through a V2 pipeline on its main branch. Developers merge several times an hour.

Last week a newer run overtook an older one, so the older commit's migration never ran and the next release failed. The company requires every merged commit to be deployed, in order, with each deployment finishing before the next begins.

What should the company change?

A Set the pipeline's execution mode to PARALLEL

B Disable the transition into the deploy stage during business hours

C Set the pipeline's execution mode to QUEUED

D Keep SUPERSEDED mode and add a manual approval before deployment

◆ PAUSE AND DECIDE

What should the company change?

A PARALLEL

Runs would no longer be skipped, but they would deploy concurrently and out of order. Exactly what a migration cannot tolerate.

B Disable the transition by day

Queues nothing and orders nothing. It just delays the same problem until evening.

C QUEUED

Newer runs wait their turn, every run completes, and they complete in order. One setting.

D SUPERSEDED plus approval

The approver cannot stop a waiting run from superseding an older one. The mode is the cause.

THE SIGNAL

'Every commit', 'in order', 'finish before the next': three phrases, one mode.

A team keeps two services, payments and search, in one GitHub repository connected through CodeConnections. Each service has its own V2 pipeline.

Every commit starts both pipelines, even when only one service changed, which wastes action minutes and causes needless production deployments. The team wants each pipeline to run only for its own service's changes, with the least effort.

What should the team do?

A Add a push trigger to each pipeline with a file-path filter for that service's folder

B Route the GitHub webhook to a Lambda function that inspects changed files and starts the right pipeline

C Split the repository into one repository per service

D Add an entry condition with a VariableCheck rule that skips the deploy stage when the service is unchanged

◆ PAUSE AND DECIDE

What should the team do?

A File-path filter per pipeline

The run never starts for the other service's changes. No code, no wasted minutes, no deployment.

B Webhook to a Lambda router

Works, but rebuilds in custom code what V2 triggers already do. The Lambda trap from module 1.

C Split the repository

Solves it by restructuring the codebase. Far more effort than a trigger filter.

D VariableCheck skip on deploy

The run still starts, still builds, still costs minutes. It hides the waste rather than removing it.

THE SIGNAL

Monorepo, one pipeline per service, needless runs: file-path filters on V2 triggers.

> During a recent incident, production was already degraded when a scheduled release started. The new deployment made diagnosis harder and extended the outage.
>
> A CloudWatch alarm named prod-health already tracks production. The company wants the pipeline to refuse to start the production stage whenever that alarm is in ALARM, with the least operational overhead.

What should a DevOps engineer configure?

A. A Lambda invoke action at the start of the stage that checks the alarm

B. A manual approval before production, with reviewers told to check the dashboard

C. EventBridge rules that disable the stage transition on ALARM and re-enable it on OK

D. An entry condition on the production stage with a CloudWatchAlarm rule and a Fail result

◆ PAUSE AND DECIDE

What should a DevOps engineer configure?

A **Lambda check action**

Works, but it is code to write, permission, and maintain, including the job-result calls. A built-in rule does this.

B **Manual approval**

Relies on a person noticing, and blocks every release, healthy or not. Not automatic.

C **EventBridge toggling the transition**

Clever and genuinely workable, but two rules and a race between them. More moving parts than needed.

D **Entry condition, CloudWatchAlarm rule, Fail**

Exactly this feature's purpose: check health before entry, refuse if unhealthy. Pure configuration.

THE SIGNAL

'Refuse to start the stage when' describes an entry condition. The alarm already exists, so it is one rule.

A pipeline stage runs a Lambda invoke action that performs a smoke test against staging. The function's logs show it completes successfully in about four seconds.

Even so, the action stays In Progress for about 20 minutes and then fails with a timeout. The function's own timeout is set to 60 seconds.

What is the most likely cause?

A The function cannot read the input artifact from the artifact bucket

B The function never calls PutJobSuccessResult, so CodePipeline receives no result

C The function's 60-second timeout is too short for a smoke test

D The pipeline is in QUEUED mode, so the action waits for an earlier run

◆ PAUSE AND DECIDE

What is the most likely cause?

A Cannot read the artifact

The function would log an access error and fail, not finish successfully in four seconds.

B No PutJobSuccessResult call

The function finishes, but CodePipeline hears nothing. After 20 minutes without a reply the action times out.

C Function timeout too short

It finished in four seconds. A 60-second limit was never reached.

D QUEUED mode

Queued runs wait at the stage entrance, not inside a running action.

THE SIGNAL

Function succeeds, action hangs, then times out at 20 minutes: the job result was never reported.

Module 2 in five lines

- V2 for triggers, execution modes, variables and stage conditions.
- SUPERSEDED skips, QUEUED orders, PARALLEL isolates.
- Events start pipelines. Polling is the legacy answer. Path filters tame monorepos.
- Entry, On Failure, On Success: health gates and automatic stage rollback without code.
- Lambda invoke actions must report a job result, or they time out at 20 minutes.

> ON THE EXAM
>
> Next, module 3: inside the build action itself. CodeBuild, the buildspec, and where secrets and caches come from.